

Comandos Básicos de Git (Control de Versiones)

Comando	Descripción	Ejemplo de uso
<code>git init</code>	Inicializa un nuevo repositorio Git en la carpeta actual.	<code>git init</code>
<code>git clone <url></code>	Clona (descarga) un repositorio remoto a tu máquina local.	<code>git clone https://github.com/usuario/proyecto.git</code>
<code>git status</code>	Muestra el estado actual de los archivos (modificados, sin seguimiento, preparados).	<code>git status</code>
<code>git add <archivo></code>	Agrega un archivo al área de preparación (staging area).	<code>git add index.html</code>
<code>git add .</code>	Agrega todos los archivos modificados al área de preparación.	<code>git add .</code>
<code>git commit -m "mensaje"</code>	Guarda los cambios en el historial del repositorio con un mensaje descriptivo.	<code>git commit -m "Agrega nueva función de login"</code>
<code>git log</code>	Muestra el historial de confirmaciones (commits).	<code>git log</code>
<code>git diff</code>	Muestra los cambios realizados en los archivos antes de confirmar.	<code>git diff</code>
<code>git branch</code>	Muestra las ramas existentes o crea una nueva.	<code>git branch nueva-rama</code>
<code>git checkout <rama></code>	Cambia a otra rama existente.	<code>git checkout main</code>

Comando	Descripción	Ejemplo de uso
<code>git merge <rama></code>	Fusiona los cambios de una rama con la rama actual.	<code>git merge desarrollo</code>
<code>git remote add origin <url></code>	Vincula el repositorio local con un repositorio remoto.	<code>git remote add origin https://github.com/usuario/proyecto.git</code>
<code>git push origin <rama></code>	Envía los commits locales al repositorio remoto.	<code>git push origin main</code>
<code>git pull origin <rama></code>	Descarga y combina los últimos cambios del remoto con tu rama local.	<code>git pull origin main</code>
<code>git reset --hard <commit></code>	Revierte el repositorio al estado de un commit específico (⚠️ Destruye cambios posteriores).	<code>git reset --hard alb2c3d</code>
<code>git rm <archivo></code>	Elimina un archivo del repositorio y del sistema.	<code>git rm archivo.txt</code>
<code>git config --global user.name "Tu Nombre"</code>	Configura tu nombre de usuario global para Git.	—
<code>git config --global user.email "tuemail@example.com"</code>	Configura tu correo electrónico global.	—

📌 Consejos útiles:

- Siempre usa `git status` antes de hacer `commit` para verificar qué archivos cambiarán.
- Utiliza mensajes de commit claros: "Corrige error de autenticación", "Agrega validación al formulario", etc.

Conceptos Básicos de Tipos de Datos y Operadores en C#

Conceptos Básicos de Tipos de Datos y Operadores en C#

1. Introducción

El lenguaje C# (C Sharp) es un lenguaje de programación moderno, orientado a objetos y fuertemente tipado. Todas las variables deben tener un tipo de dato definido que indique qué valores puede almacenar y qué operaciones se pueden realizar.

Tipos de Datos en C#

En C#, los tipos de datos se dividen en **dos categorías principales**:

◆ A. Tipos de Valor (Value Types)

Guardan directamente el valor en memoria.

Tipo	Descripción	Tamaño	Ejemplo
byte	Entero positivo	8 bits	byte edad = 20;
int	Entero con signo	32 bits	int cantidad = 150;
long	Entero largo	64 bits	long poblacion = 8000000;
float	Decimal (precisión simple)	32 bits	float pi = 3.14f;
double	Decimal (precisión doble)	64 bits	double radio = 2.718281;
decimal	Decimal de alta precisión	128 bits	decimal precio = 99.99m;
char	Carácter Unicode	16 bits	char letra = 'A';
bool	Valor lógico	1 bit	bool activo = true;

◆ B. Tipos de Referencia (Reference Types)

Guardan una **referencia a un objeto** en memoria (no el valor directamente).

Tipo	Descripción	Ejemplo
string	Cadena de texto	<code>string nombre = "Marche";</code>
object	Tipo base de todos los tipos	<code>object dato = 42;</code>
array	Conjunto de valores del mismo tipo	<code>int[] numeros = {1,2,3};</code>
class	Define una clase (objeto complejo)	<code>class Persona { public string Nombre; }</code>

◆ C. Tipos Anulables (Nullable Types)

Permiten que los tipos de valor almacenen `null`:

```
int? edad = null;
```

⚙ 3. Operadores en C#

Los **operadores** permiten realizar operaciones matemáticas, comparaciones y decisiones lógicas.

📊 3.1 Operadores Matemáticos

Operador	Descripción	Ejemplo	Resultado
+	Suma	<code>5 + 2</code>	7
-	Resta	<code>5 - 2</code>	3
*	Multiplicación	<code>5 * 2</code>	10
/	División	<code>5 / 2</code>	2
%	Módulo (resto)	<code>5 % 2</code>	1

🔍 3.2 Operadores de Comparación

Operador	Significado	Ejemplo	Resultado
<code>==</code>	Igual a	<code>5 == 5</code>	true

Operador	Significado	Ejemplo	Resultado
!=	Distinto de	5 != 3	true
>	Mayor que	8 > 3	true
<	Menor que	3 < 8	true
>=	Mayor o igual	5 >= 5	true
<=	Menor o igual	4 <= 5	true

⚡ 3.3 Operadores Lógicos

Operador	Descripción	Ejemplo	Resultado
&&	AND (Y lógico)	true && false	false
^		^	OR (O lógico)
!	NOT (Negación)	!true	false

🔗 3.4 Conectivos Lógicos (Relación con lógica proposicional)

Conectivo	Significado lógico	Operador en C#	Ejemplo lógico
$\wedge$	Conjunción	&&	$P \wedge Q$
$\vee$	Disyunción	^	
$\neg$	Negación	!	$\neg P$
$\rightarrow$	Implicación (si... entonces)	No tiene operador directo (se usa if)	if (P) then Q
$\leftrightarrow$	Bicondicional	No tiene operador directo	$P == Q$

🔧 3.5 Operadores de Incremento y Decremento

Operador	Descripción	Ejemplo	Resultado
++i	Incremento antes de usar el valor	int i = 1; Console.WriteLine(++i);	Imprime 2
i++	Incremento después de usar el valor	int i = 1; Console.WriteLine(i++);	Imprime 1

Operador	Descripción	Ejemplo	Resultado
--i	Decremento antes de usar el valor	<code>int i = 2; Console.WriteLine(--i);</code>	Imprime 1
i--	Decremento después de usar el valor	<code>int i = 2; Console.WriteLine(i--);</code>	Imprime 2

Tablas de verdad y operadores se incluyen a continuación.

Operadores Matemáticos

Operador	Descripción	Ejemplo	Resultado
+	Suma	<code>5 + 2</code>	7
-	Resta	<code>5 - 2</code>	3
*	Multiplicación	<code>5 * 2</code>	10
/	División	<code>5 / 2</code>	2
%	Módulo	<code>5 % 2</code>	1

Tablas de Verdad

AND (&):

P	Q	P & Q
T	T	T
T	F	F
F	T	F
F	F	F

OR (||):

P	Q	P Q
T	T	T
T	F	T
F	T	T
F	F	F

NOT (!):

P	!P
T	F
F	T